

DOCUMENTO DE PLANEACIÓN SCRUM

Irma Yaneht Arias García

1.1 Roles Asignados

- **Product Owner (PO):** Define los requisitos del negocio, prioriza el Product Backlog, acepta los entregables de cada Sprint y asegura que el producto aporte valor.
- **Scrum Master (SM):** Facilita las ceremonias Scrum, elimina impedimentos del equipo, asegura el cumplimiento de la metodología y la calidad del proceso.
- **Developer(s):** Responsable(s) de la implementación del código, pruebas unitarias/integración, aplicación de patrones de diseño y documentación técnica en Node.js y MySQL.

1.2 Sprints Definidos

- **Sprint 1 (Días 1 y 2):** Fundación y Core Business: Análisis, arquitectura, configuración de la base de datos MySQL, modelos con validaciones, repositorios y gestión de clientes.
- **Sprint 2 (Días 3 y 4):** Funcionalidades Avanzadas: Planes de entrenamiento, contratos automáticos, seguimiento físico, nutrición y control financiero con transacciones SQL reales.

1.3 Épicas del Proyecto

- **Épica 1: Fundación y Base de Datos (Días 1-2):** Configuración del entorno, estructura de carpetas (src/models, src/repositories, src/services, src/comands), conexión con MySQL y clases base.
- **Épica 2: Core Business (Días 2-3):** Desarrollo de módulos de clientes, planes de entrenamiento y contratos automáticos vinculados por ID.
- **Épica 3: Funcionalidades Avanzadas y Transacciones (Días 3-4):** Implementación de seguimiento físico, nutrición, control financiero y transacciones SQL críticas con rollbacks.
- **Épica 4: Entrega Final (Día 4):** Consolidación en menu.js, pruebas de consola con inquirer/chalk, documentación y video demo.

2. TAREAS DETALLADAS CON SUBTAREAS

- **TASK 01 — Configuración y Conexión MySQL (Día 1):**
 - Instalación de dependencias npm (mysql2, dotenv, inquirer, chalk).
 - Creación del archivo .env y configuración del pool de conexiones en la carpeta /config.
 - Creación de esquemas iniciales en MySQL.

- **TASK 02 — Módulo de Clientes y Modelos POO (Día 2):**
 - Creación de la clase ClientModel en /models con validaciones de datos (edades, formatos de correo, campos requeridos).
 - Implementación del patrón *Repository* (clientRepository.js) en la carpeta /repositories.
 - Desarrollo del menú por consola en src/comands/menu.js para el CRUD de clientes.
- **TASK 03 — Planes y Contratos Automáticos (Día 3):**
 - Creación de modelos para planes y contratos en /models.
 - Programación de la lógica para que al asignar un plan se genere el contrato de forma automática en MySQL.
- **TASK 04 — Transacciones y Finanzas (Día 4):**
 - Implementación de control financiero (ingresos y egresos).
 - Uso de transacciones SQL reales (BEGIN TRANSACTION, COMMIT, ROLLBACK) para garantizar la consistencia ante cancelaciones o pagos.

3. REQUERIMIENTOS E HISTORIAS DE USUARIO

ID	Historia de Usuario	Criterios de Aceptación
HU-01	Como entrenador, quiero gestionar clientes (crear, listar, actualizar, eliminar) para mantener la base de datos organizada.	El sistema valida campos obligatorios y formatos correctos antes de insertar en MySQL.
HU-02	Como administrador, quiero que al asignar un plan de entrenamiento a un cliente se genere un contrato automáticamente.	El contrato calcula fechas de inicio/fin, precio y condiciones vinculándose al cliente en una sola transacción.
HU-03	Como gestor financiero, quiero registrar ingresos y egresos usando transacciones reales de MySQL para evitar inconsistencias.	Si ocurre un fallo en el registro financiero, se ejecuta un ROLLBACK automático en la base de datos.

4. HERRAMIENTA DE SEGUIMIENTO Y EVIDENCIAS

- **Herramienta de Gestión:** ClickUp / GitHub Projects.

- **Control de Versiones:** Commits estructurados con Conventional Commits (feat:, fix:, docs:).
- **Estructura del Proyecto en VS Code:**

Plaintext

FitCoreCLI-Yani/

```
├── src/
│   ├── comands/
│   │   └── menu.js
│   ├── config/
│   ├── models/
│   │   ├── client.js
│   │   ├── contract.js
│   │   └── plan.js
│   ├── repositories/
│   └── services/
├── docs/
│   └── planificación-Scrum.pdf
├── .env
├── .gitignore
├── package.json
└── README.md
```